

Technical Report: LLM Monitoring System

1. Project Objective

The goal of this project is to build a simplified but functional LLM Monitoring System that:

- collects public LLM benchmark and operational data from at least two sources
- normalizes heterogeneous metrics into a comparable 0-100 scale
- stores collection history in SQLite
- recommends models for different enterprise profiles
- exposes the system through a FastAPI API
- provides a Streamlit dashboard and a digest report

The project was designed as an MVP for a technical challenge, so the focus was on correctness, simplicity, explainability, and demo readiness.

2. System Architecture

The application is organized into the following layers:

- ``collectors/``: fetch raw data from public sources
- ``services/``: business logic for collection, persistence, recommendation, and dashboard preparation
- ``utils/``: normalization, scoring, license filtering, and report generation
- ``api/``: FastAPI routes
- ``dashboard/``: Streamlit application
- ``data/``: SQLite database
- ``reports/``: generated markdown reports

Main flow

1. A collection run is triggered manually, via API, or by the scheduler.
2. Collectors retrieve public LLM data from Hugging Face and Vellum.
3. Raw records are normalized and stored in SQLite.
4. The system compares the latest run with the previous one to detect new models.
5. A recommendation engine computes profile-based scores.
6. Results are served through FastAPI and visualized in Streamlit.
7. A markdown digest report is generated from the database.

3. Technical Choices And Justification

Python 3.11+

Python was the natural choice because the challenge requires Python and the ecosystem is strong for APIs, data processing, and dashboards.

SQLite

SQLite was chosen because:

- it requires zero infrastructure
- it is easy to run locally on macOS or Windows
- it stores everything in a single file
- it is sufficient for an MVP and a technical demo

This choice reduces setup complexity and makes the project easy to review and reproduce.

SQLAlchemy

SQLAlchemy was used instead of raw `sqlite3` because the system contains multiple related tables:

- collection runs
- sources
- models
- model metrics
- recommendation results

Using SQLAlchemy makes these relationships easier to manage and easier to explain clearly.

FastAPI

FastAPI was chosen because:

- it is quick to implement
- request and response validation is built in
- Swagger documentation is generated automatically
- it is clean for interview demos

Streamlit

Streamlit was chosen because it is the fastest way to build a simple analytics dashboard without frontend development.

Weighted scoring formula

The recommendation system uses a weighted average of normalized metrics. This was chosen because:

- it is transparent
- it is easy to explain orally
- it supports multiple enterprise profiles cleanly
- it avoids black-box behavior

4. Data Sources

The system currently uses two public sources:

1. Hugging Face Open LLM Leaderboard

Used for:

- model name
- benchmark intelligence score
- license metadata
- update information when available

2. Vellum LLM Leaderboard

Used for:

- input cost
- output cost
- speed
- latency
- context window

These two sources complement each other well because one is benchmark-oriented and the other is operational/performance-oriented.

5. Database Design

The SQLite schema contains the following main tables:

`collection\_runs`

Stores each full collection execution with timestamps and status.

`sources`

Stores metadata about data sources.

`models`

Stores unique model identities with normalized names.

`model\_metrics`

Stores the metrics collected for one model from one source during one run. This table stores both raw and normalized values.

`recommendation\_results`

Stores optional saved recommendation outputs.

This schema supports historical comparisons, reporting, and recommendation persistence.

6. Normalization And Scoring

The challenge requires heterogeneous metrics to be compared fairly. The project solves this by normalizing metrics to a 0-100 scale.

Higher-is-better metrics

- intelligence score
- tokens per second
- context window

Lower-is-better metrics

- input price per 1M tokens
- output price per 1M tokens
- time to first token

Missing values

Missing values are never fabricated. They are stored as `NULL`.

During scoring:

- missing metrics are skipped
- they are not treated as zero

This prevents unfair penalties when a public source does not expose a metric.

7. Enterprise Profiles

The project supports these profiles:

- `coding\_dev`
- `reasoning\_analysis`
- `enterprise\_agents`
- `long\_context\_rag`
- `minimum\_cost`

Each profile uses a different weight configuration. For example:

- `coding\_dev` emphasizes intelligence, speed, and latency
- `long\_context\_rag` emphasizes context window
- `minimum\_cost` emphasizes input and output prices

8. Commercial Use Filter

A conservative license filter is implemented.

Rule:

- clearly permissive licenses are allowed
- clearly restrictive or unclear licenses are excluded when `commercial\_use=true`

This is a practical compliance guard for an MVP.

9. New Model Detection

A model is considered new if it appears in the latest completed run but not in the previous completed run.

This is implemented through set comparison of model names between runs. The first run marks all models as new because no baseline exists yet.

10. Challenges Encountered

HTML parsing dependency

The Vellum collector required `lxml` for reliable HTML table parsing.

Missing and inconsistent source data

Public leaderboards do not expose every metric for every model. This required:

- null-safe storage
- robust normalization
- weighted scoring that ignores missing metrics

NaN handling

Some parsed HTML values were represented as `NaN` instead of `None`. This created incorrect recommendation scores at first. The fix was to treat `NaN` as missing data during parsing, normalization, and scoring.

Conservative cross-source matching

Model names differ slightly between sources. To avoid false merges, the MVP uses simple normalized-name matching instead of fuzzy matching.

11. Current Limitations

- cross-source model matching is conservative
- some models still remain source-specific rows
- public sources may change their HTML or structure over time
- some metrics remain unavailable and are stored as null
- the scheduler is simple and local, not production-grade

12. Possible Improvements

- improve cross-source model matching with alias mapping
- add more public sources
- add JSON report export in addition to markdown
- add PDF export
- deploy the API and dashboard publicly

- replace the simple scheduler with APScheduler or cron
- add automated tests for collectors and scoring

13. Screenshots To Include In The PDF

Use these screenshots in your final PDF:

1. Streamlit dashboard overview
2. Swagger UI endpoint list
3. `/recommend` API response
4. Generated markdown report preview
5. Optional: project folder structure in the editor

14. Conclusion

This project delivers a complete beginner-friendly MVP for LLM monitoring. It covers data collection, normalization, scoring, storage, API access, dashboard visualization, reporting, and basic scheduling. The design favors simplicity and explainability while still satisfying the main technical requirements of the challenge.